

Email Content Analysis for Detecting Spam Messages

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Venkata Sai Krishna (2520030027)
Pujith Pavan Kumar (2520030029)
Ajai Manikanta (2520030320)

Team Number: 03

Section: 07

Under the guidance of

S. Madhavi

Professor , CSE



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

The Mini Project Report entitled “**Email Content Analysis for Detecting Spam Messages**” is a record of Bonafide work of **Venkata Sai Krishna – 2520030027, Pujith Pavan Kumar – 2520030029, Ajai Manikanta – 2520030320** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Venkata Sai Krishna – 2520030027

Pujith Pavan Kumar – 2520030029

Ajai Manikanta – 2520030320

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled “**Email Content Analysis for Detecting Spam Messages**” is a bonafide work done and submitted by **Venkata Sai Krishna – 2520030027, Pujith Pavan Kumar – 2520030029, Ajai Manikanta – 2520030320** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We would like to express our sincere thanks to everyone who supported us in one way or another in completing this project.

We are deeply grateful to our project guide, S. Madhavi, Professor, Department of Computer Science and Engineering, for her constant support, encouragement and guidance throughout this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)**, Head of the Department of Computer Science and Engineering, for providing us with the facilities and resources needed to complete this project-based lab.

We thank all the teaching and non-teaching staff members who helped us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Venkata Sai Krishna – 2520030027

Pujith Pavan Kumar – 2520030029

Ajai Manikanta – 2520030320

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	17
8.	Conclusion & Future Scope	21
9.	References	22

1.ABSTRACT

Spam emails are a common problem for every email user. They waste time and may contain fake offers, prize messages, account verification scams and harmful links, so checking the content of an email is an important step in protecting users. This project, “Email Content Analysis for Detecting Spam Messages”, is a Java-based DSA project with a web interface. It analyses the content of an email and classifies it as SPAM or NOT SPAM.

The system first loads the emails and preprocesses the text. Spam keywords and suspicious patterns are then searched using four string matching algorithms: KMP (with the LPS array), Rabin-Karp (with a rolling hash), the Z-Algorithm (with the Z-array) and Aho-Corasick (with a trie and failure links). Bitmask Dynamic Programming represents the detected features as bits and evaluates the possible combinations to get the best feature score. Edmonds-Karp Maximum Flow uses BFS to find augmenting paths in a residual network, and Greedy Set Cover Approximation repeatedly selects the rule that covers the most uncovered features. Randomized hashing uses a randomly selected base for the hash calculation, and parallel processing uses multiple Java tasks to process emails at the same time.

The detected features are combined into a spam score, and this score decides whether the email is marked SPAM or NOT SPAM. The system also shows the spam category, confidence score, risk level and the detected and suspicious URLs. The results, the algorithm comparison and a detailed report are shown through the web interface. The final classification uses a rule/feature-based scoring approach and is not a machine learning model.

Team Members:

1. Venkata Sai Krishna, 2520030027
2. Pujith Pavan Kumar, 2520030029
3. Ajai Manikanta, 2520030320

2. INTRODUCTION

Email is one of the most widely used ways of communication. Students, companies and government offices all depend on it for daily work. Because it is so common and costs almost nothing to use, it is also a favourite target for spammers.

Spam emails are unwanted messages that are sent to many people at the same time. They come in many forms, such as fake offers and discounts, prize or lottery messages, account verification scams that ask for passwords, financial scams that ask for money or bank details, and messages with suspicious links. These emails waste time and storage, and some of them can cause real financial loss or steal personal information.

Checking only the sender of an email is not enough, because the sender address can be faked or can look genuine. The content of the email gives better clues. Spam messages often use words like “free”, “winner” or “urgent”, repeat certain phrases, and include links to unknown websites. This is why the content of the email needs to be analysed.

The motivation for this project was to apply the algorithms learned in the DSA-3 course to a real problem. String matching is a natural fit for finding suspicious words and patterns in text. KMP, Rabin-Karp and the Z-Algorithm search for one pattern in the text, while Aho-Corasick searches for many keywords together using a trie with failure links. The other algorithms in the project also have a role. Bitmask Dynamic Programming is used for feature-score calculation, Edmonds-Karp Maximum Flow and Greedy Set Cover Approximation are applied in the analysis, and randomized hashing and parallel email processing are demonstrated.

In the system, an email is loaded and preprocessed, the algorithms look for spam keywords, suspicious patterns and URLs, and the findings are combined into a spam score. Based on this score the email is classified as SPAM or NOT SPAM, and the system also gives the spam category, confidence score and risk level. The current system uses a rule/feature-based approach and does not use machine learning. Everything is available through a web interface, which also shows the algorithm comparison and a detailed report.

The rest of the report contains the literature survey, the problem statement and objectives, the hardware and software requirements, the implementation of four main modules, the results and discussion, and finally the conclusion with future scope and the references.

3. LITERATURE SURVEY

Shirvani and Ghasemshirazi, 2025 [1]. This study uses zero-shot learning for email spam detection. BERT is used to preprocess the email and extract important information from its content, and the FLAN-T5 language model then classifies the email without being trained on labelled spam examples for this task. It is relevant to our work because it also analyses email content, but it depends on large pre-trained language models. Our project does not use any language model and works with string matching and other DSA algorithms.

Hotoğlu, Sen and Can, 2025 [2]. This paper studies adversarial attacks on six deep learning spam filters using the SpamAssassin, Enron Spam and TREC 2007 datasets. The attacks change words, characters, sentences or whole paragraphs (including AI-generated text) so that spam can pass the filter. It shows that spammers can change their text to escape detection, which is also a limitation for any keyword-based method like ours. Our project does not study attacks and does not use deep learning.

Ali and Abdullah, 2025 [3]. The authors propose a hybrid approach that combines machine learning and deep learning for email spam detection, and they test it on the Ling-Spam, Enron and SpamAssassin datasets. Their work also deals with spam that hides words by replacing characters. It is relevant because it shows how spam text is disguised. Their method needs training data, whereas our project uses a rule/feature-based score.

Tusher, Ismail and Mat Raffei, 2025 [4]. This is a review paper on deep learning methods for email spam classification, with a focus on CNN. It gives an overview of how deep learning models are used on email text and helps in understanding the current direction of research. Our project is not a review and does not use deep learning.

Kshirsagar, Rathi and Ryan, 2025 [5]. The authors compare five traditional machine learning and five deep learning spam classifiers with a new meta-learner on the Enron-Spam and TREC 2007 datasets. Their aim is also to make the spam detection more interpretable. Our project does not train or compare such models and has no labelled-dataset evaluation, so we do not compare results with this work.

Datasets used in these studies [6], [7]. The studies above commonly use public email collections such as the SpamAssassin corpus [6], Enron-Spam and TREC corpora [7]. We have not used these datasets for any result in this report, but they can be used for testing in future work.

Comparison with our project. The recent research above mostly uses machine learning, deep learning or large language models, which are trained on datasets. Our project does not train a machine learning model. It focuses on showing how multiple DSA algorithms (string matching, dynamic programming, maximum flow, greedy and hashing methods) can be combined for email content analysis. We do not claim that it performs better than any of these studies.

4. Problem Statement and Objectives

Problem Statement: A clear and specific articulation of the problem, outlining its importance.

Detect whether an email is SPAM or NOT SPAM by analysing its content.

Problem Description: A concise explanation of the issue or challenge that needs to be solved.

Checking each email by hand for spam words, patterns and suspicious links is slow and easy to get wrong, so the content must be scanned automatically and the result explained clearly.

Objectives:

Identify the Project's Purpose:

The project was started to apply DSA-3 algorithms to a real problem, detecting spam emails from their content. The objectives are:

1. To load email messages and preprocess their content.
2. To detect spam-related keywords and phrases.
3. To apply the KMP algorithm for pattern matching.
4. To apply the Rabin-Karp algorithm.
5. To apply the Z-Algorithm.
6. To apply the Aho-Corasick algorithm for searching many keywords together.
7. To use Bitmask Dynamic Programming for feature-score calculation.
8. To apply Edmonds-Karp Maximum Flow.
9. To apply Greedy Set Cover Approximation.
10. To demonstrate randomized hashing.
11. To demonstrate parallel email processing.
12. To compare the implemented algorithms.
13. To calculate the spam score of an email.
14. To classify emails as SPAM or NOT SPAM.
15. To identify the spam category.
16. To calculate the confidence and risk level.
17. To detect URLs and suspicious URLs.
18. To generate detailed analysis reports.
19. To provide a web-based interface.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Computer / laptop
- Processor capable of running JDK 17
- Minimum 4 GB RAM
- Keyboard and mouse
- Standard display
- Sufficient storage for project files

Software Requirements:

- Java
- JDK 17 or later
- Visual Studio Code
- HTML
- CSS
- JavaScript
- Web browser
- Windows / Linux
- Text files for email and spam keyword data

6. IMPLEMENTATION

Module 1: String Matching using KMP

Purpose: To check whether a spam keyword or phrase is present in the email text, without repeating unnecessary comparisons.

Input: The email text and a pattern (spam keyword), both given as strings.

How it searches: KMP first builds the LPS (longest proper prefix which is also a suffix) array for the pattern. When a mismatch happens, the LPS array tells how far the pattern can be shifted, so the text pointer never moves backwards. The search returns true as soon as the whole pattern is matched.

Role in spam detection: It tells whether a spam keyword occurs in the email, and the keywords found are used in the spam score.

Complexity: $O(n + m)$ for a text of length n and a pattern of length m .

Code:

```
public class KMP {
    // Builds the LPS (Longest Prefix Suffix) array
    private static int[] buildLPS(String pattern) {
        int[] lps = new int[pattern.length()];
        int length = 0;
        int i = 1;
        while (i < pattern.length()) {
            if (pattern.charAt(i) == pattern.charAt(length)) {
                length++;
                lps[i] = length;
                i++;
            }
            else {
                if (length != 0) {
                    length = lps[length - 1];
                }
                else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
        return lps;
    }
    // Returns true if the pattern is found in the text
    public static boolean search(String text, String pattern) {
        if (pattern.length() == 0) {
            return true;
        }
        int[] lps = buildLPS(pattern);
        int i = 0;
        int j = 0;
        while (i < text.length()) {
            if (text.charAt(i) == pattern.charAt(j)) {
                i++;
                j++;
                if (j == pattern.length()) {
                    return true;
                }
            }
        }
    }
}
```

```

    }
    else {
        if (j != 0) {
            j = lps[j - 1];
        }
        else {
            i++;
        }
    }
}
return false;
}
}

```

Module 2: Aho-Corasick

Purpose: To search for many spam keywords in the email text in a single pass.

Trie: All the keywords are first inserted into a trie, where each path from the root spells a keyword. The last node of a keyword stores that keyword as an output.

Failure links: For every node, a failure link points to the longest proper suffix that is also a prefix of some keyword. The links are built using BFS, and the outputs of the failure node are added to each node. On a mismatch the search follows the failure link instead of starting again.

Role in spam keyword detection: The search method scans the email text once and returns the list of keywords found, each keyword only once.

Complexity: $O(n + m + z)$, where n is the text length, m is the total length of all keywords and z is the number of matches.

Code:

```

import java.util.*;
public class AhoCorasick {
    private static class Node {
        Map<Character, Integer> children = new HashMap<>();
        int failure = 0;
        List<String> outputs = new ArrayList<>();
    }
    private final List<Node> trie = new ArrayList<>();
    public AhoCorasick(String[] patterns) {
        trie.add(new Node());
        for (String pattern : patterns) {
            int current = 0;
            for (char ch : pattern.toCharArray()) {
                if (!trie.get(current).children.containsKey(ch)) {
                    trie.get(current).children.put(ch, trie.size());
                    trie.add(new Node());
                }
                current = trie.get(current).children.get(ch);
            }
            trie.get(current).outputs.add(pattern);
        }
        buildFailureLinks();
    }
    private void buildFailureLinks() {
        Queue<Integer> queue = new LinkedList<>();
        for (int next : trie.get(0).children.values()) {
            trie.get(next).failure = 0;
            queue.add(next);
        }
    }
}

```

```

    }
    while (!queue.isEmpty()) {
        int current = queue.poll();
        for (Map.Entry<Character, Integer> entry : trie.get(current).children.entrySet()) {
            char ch = entry.getKey();
            int next = entry.getValue();
            queue.add(next);
            int failure = trie.get(current).failure;
            while (failure != 0 && !trie.get(failure).children.containsKey(ch)) {
                failure = trie.get(failure).failure;
            }
            if (trie.get(failure).children.containsKey(ch)) {
                trie.get(next).failure = trie.get(failure).children.get(ch);
            } else {
                trie.get(next).failure = 0;
            }
            trie.get(next).outputs.addAll(trie.get(trie.get(next).failure).outputs);
        }
    }
}

public List<String> search(String text) {
    List<String> foundPatterns = new ArrayList<>();
    if (text == null) return foundPatterns;
    int current = 0;
    for (char ch : text.toCharArray()) {
        while (current != 0 && !trie.get(current).children.containsKey(ch)) {
            current = trie.get(current).failure;
        }
        if (trie.get(current).children.containsKey(ch)) {
            current = trie.get(current).children.get(ch);
        }
        for (String pattern : trie.get(current).outputs) {
            if (!foundPatterns.contains(pattern)) {
                foundPatterns.add(pattern);
            }
        }
    }
    return foundPatterns;
}
}

```

Module 3: Bitmask Dynamic Programming

Purpose: To calculate the best feature score of an email from the scores of its spam features.

Features as bits: Each feature is represented by one bit of a mask, so a mask such as 101 means that the first and third features are selected.

Combinations: The code goes through every mask from 0 to $2^n - 1$ and adds up the scores of the features whose bits are set, so all combinations of features are evaluated.

Best feature score: Only the masks that select at most a given limit of features are considered, and the highest score among them is kept as the best score.

Contribution to spam analysis: The best score is shown as the “Bitmask DP Score” in the analysis output and is one of the inputs for the spam analysis.

Complexity: $O(2^n \cdot n)$ for n features.

Code:

```

public class BitmaskDP {
    public static int findBestScore(int[] scores, int limit) {

```

```

    int n = scores.length;
    int totalMasks = 1 << n;
    int bestScore = 0;
    for (int mask = 0; mask < totalMasks; mask++) {
        int currentScore = 0;
        int selectedCount = 0;
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) {
                currentScore += scores[i];
                selectedCount++;
            }
        }
        if (selectedCount <= limit &&
            currentScore > bestScore) {
            bestScore = currentScore;
        }
    }
    return bestScore;
}
}

```

Module 4: Edmonds-Karp Maximum Flow

Purpose: To calculate the maximum flow in a flow network using the Edmonds-Karp algorithm.

Network representation: The network is stored as a capacity matrix, and a second matrix keeps the flow sent through each edge.

Residual graph: The residual capacity of an edge is its capacity minus the flow on it. A reverse flow is also stored, so that flow can be cancelled if needed.

BFS and augmenting path: BFS is used to find the shortest path from the source to the sink that still has residual capacity. The smallest residual capacity on this path (the bottleneck) is pushed as flow, and the flows are updated. This is repeated until no path is left.

Maximum flow: When no augmenting path remains, the total pushed flow is the maximum flow.

How it is used in the project: The maximum flow value is calculated for each email and is shown as “Edmonds-Karp Flow” in the analysis output.

Complexity: $O(V \cdot E^2)$ for V vertices and E edges.

Code:

```

import java.util.*;
public class EdmondsKarp {
    private int[][] capacity;
    private int[][] flow;
    private int n;
    public EdmondsKarp(int[][] capacity) {
        this.capacity = capacity;
        this.n = capacity.length;
        this.flow = new int[n][n];
    }
    public int maxFlow(int source, int sink) {
        int totalFlow = 0;
        while (true) {
            int[] parent = new int[n];
            Arrays.fill(parent, -1);
            parent[source] = source;
            Queue<Integer> queue = new LinkedList<>();
            queue.add(source);

```

```

// BFS to find augmenting path
while (!queue.isEmpty() && parent[sink] == -1) {
    int current = queue.poll();
    for (int next = 0; next < n; next++) {
        int residualCapacity =
            capacity[current][next]
            - flow[current][next];
        if (parent[next] == -1
            && residualCapacity > 0) {
            parent[next] = current;
            queue.add(next);
        }
    }
}
// No augmenting path
if (parent[sink] == -1) {
    break;
}
// Find bottleneck capacity
int pathFlow = Integer.MAX_VALUE;
int current = sink;
while (current != source) {
    int previous = parent[current];
    pathFlow = Math.min(
        pathFlow,
        capacity[previous][current]
        - flow[previous][current]
    );
    current = previous;
}
// Update flow
current = sink;
while (current != source) {
    int previous = parent[current];
    flow[previous][current] += pathFlow;
    flow[current][previous] -= pathFlow;
    current = previous;
}
totalFlow += pathFlow;
}
return totalFlow;
}
}

```

7. RESULTS & DISCUSSION

Screenshot 1: Main Email Menu

```
=====
EMAIL SPAM ANALYSIS SYSTEM
=====
Total Emails: 14937 across 2000 accounts

Enter Email ID: user500@gmail.com

=====
USER: user500@gmail.com (5 messages)
=====
1. Compose & Analyze New Message
2. Display Existing Messages
3. Run Full Spam Analysis
4. Algorithm Comparison
0. Back to Main Menu
-----
Enter your choice: █
```

This screen shows the console menu after the user ID user500@gmail.com is entered. The system reports 14937 emails across 2000 accounts, and this user has 5 messages. The menu gives the options to compose and analyze a new message, display existing messages, run the full spam analysis and compare the algorithms. It is the starting point for all the features of the project.

Screenshot 2: Existing Email Display

```
Enter your choice: 2

--- Stored Messages (5) ---
Message ID: 1
Subject : Team lunch this Saturday
Content : We are planning a team lunch this Saturday. Let me know your food preference by Thursday.
-----
Message ID: 2
Subject : Project files updated
Content : I uploaded the latest project files to the shared folder. Please review them and share your feedback.
-----
Message ID: 3
Subject : Lowest loan rates
Content : Lowest loan rates guaranteed. Get up to $50,000 with no credit check and instant approval. Apply at http://claim-now.xyz/win.
-----
Message ID: 4
Subject : Internship update
Content : How is your internship going? Let me know if you need any help with the project report.
-----
Message ID: 5
Subject : Meeting rescheduled
Content : Today's meeting has been moved to 9 AM on Wednesday. Kindly confirm your availability.
-----
```

This screen shows the 5 stored messages of user500@gmail.com with their message ID, subject and content. Messages 1, 2, 4 and 5 are normal messages, while message 3 offers a loan with no credit check and contains a link. Showing the original text first helps to compare it with the analysis result.

Screenshot 3: Email Analysis Result

```
-----
Message #5 [user500@gmail.com]
Subject: Meeting rescheduled
Body : Today's meeting has been moved to 9 AM on Wednesday. Kindly confirm your availability.
-----
1. KMP Search : 0 matches []
2. Rabin-Karp Search : 0 matches []
3. Z-Algorithm Search : 0 matches []
4. Aho-Corasick Search : 0 matches []
5. Bitmask DP Score : 0 (Evaluated 0 features)
6. Edmonds-Karp Flow : 20 max flow capacity
7. Greedy Set Cover : 0 rules selected []
8. Randomized Hash : 0xDF3F2775
Category : NOT SPAM | Risk Level: LOW
Confidence: 0.00% | URLs Found: 0 (0 suspicious)
VERDICT : [NOT SPAM] (Spam Score: 0 / Threshold: 8)
-----

=====
FINAL EMAIL REPORT
=====
Total Emails : 5
Spam Emails : 1
Not Spam Emails : 4
Spam Percentage : 20.00%
=====
Spam Message IDs : [3]
Not Spam Message IDs : [1, 2, 4, 5]

Statistical Breakdown:
Total Messages : 5
Spam Messages : 1 (20.00%)
Legitimate : 4

Category Distribution:
- NOT SPAM : 4
- GENERAL SPAM : 1

Top Detected Spam Keywords:
1. guaranteed -> 1 occurrences
2. claim now -> 1 occurrences
Analysis report saved to: data/spam_analysis_report.txt
Parallel Processing: 5 emails processed concurrently on 12 CPU threads.
Email analysis completed.
```

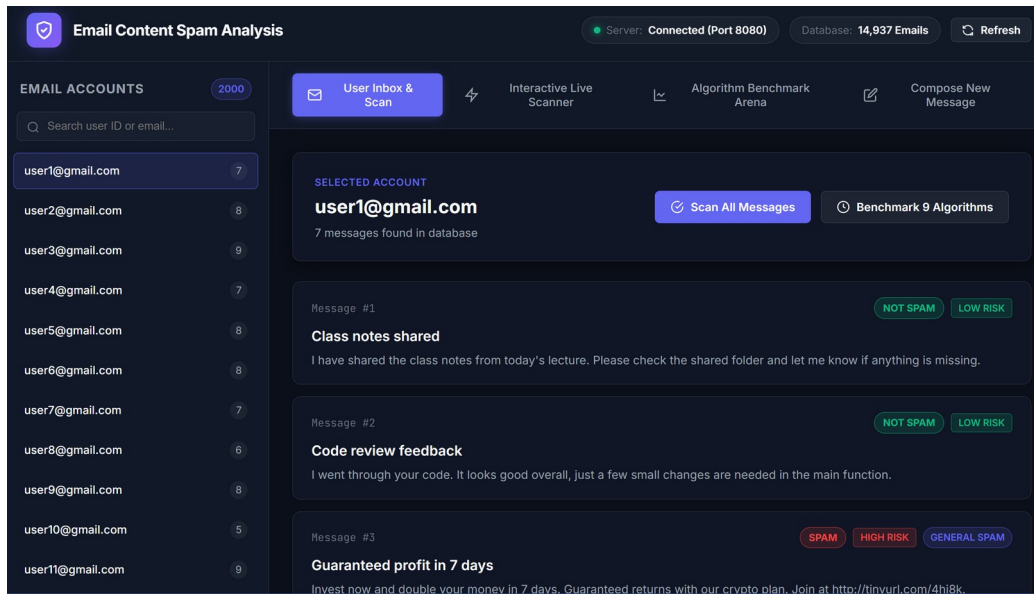
This screen shows the result of the full spam analysis. For message 5 it lists the output of all eight algorithms, the category, risk level, confidence, URLs found and the verdict (NOT SPAM, spam score 0 against threshold 8). The final report shows 1 spam email out of 5 (20.00%), the spam message ID 3, the category distribution and the top detected keywords, and says that the report is saved to a file and that the emails were processed concurrently on 12 CPU threads.

Screenshot 4: Algorithm Comparison

```
=====
                        ALGORITHM COMPARISON
=====
User Account      : user500@gmail.com
Messages Tested   : 5 emails
Keywords Tested   : 212 patterns
Keywords Found    : 2 unique detected in user500@gmail.com's emails
=====
Algorithm         Category          Complexity    Time (ms)    Result Summary
-----
KMP                String Matching    O(N + M)      32.734 ms    2 matches
Rabin-Karp         String Matching    O(N + M) avg   12.164 ms    2 matches
Z-Algorithm        String Matching    O(N + M)      20.125 ms    2 matches
Aho-Corasick       Multi-Pattern Search O(N + sum(M))  9.502 ms     2 matches
Bitmask DP         Dynamic Programming O(2^N * N)     4.002 ms     Optimal score = 10
Edmonds-Karp        Network Flow       O(V * E^2)     0.036 ms     Max Flow = 22
Greedy Set Cover    Approximation      O(|U| * |S|)   15.975 ms     Selected 43 rules
Randomized Hashing Randomized Algorithm O(N)           0.215 ms     Hashed 5 emails
Parallel Processing Concurrency      O(N / P)       2.229 ms     Processed on 12 threads (5 emails)
=====
Fastest String Matcher : Aho-Corasick (9.502 ms) [Single-pass Trie traversal]
Overall Lowest Latency : Edmonds-Karp (0.036 ms)
Dataset Evaluated      : 5 messages of user500@gmail.com against 212 patterns
Sample Detected Words  : [guaranteed, claim now]
Comparison report saved to: data/algorithm_comparison_report.txt
```

This screen compares the algorithms on the 5 messages of user500@gmail.com against 212 keyword patterns. For each algorithm it shows the category, complexity, time in milliseconds and a result summary. In this run all four string matching algorithms found the same 2 matches and Aho-Corasick was the fastest string matcher. The times are from one run only and are not a formal benchmark.

Screenshot 5: Web Interface



This is the web interface of the project, connected to the server on port 8080 with a database of 14,937 emails. The left side lists the 2000 email accounts, and the tabs give User Inbox & Scan, Interactive Live Scanner, Algorithm Benchmark Arena and Compose New Message. For user1@gmail.com each message is shown with labels such as NOT SPAM / LOW RISK or SPAM / HIGH RISK / GENERAL SPAM.

8. CONCLUSION & FUTURE SCOPE

In this project we built a Java-based DSA project with a web interface that analyses the content of an email and classifies it as SPAM or NOT SPAM. The emails are first loaded and preprocessed. Then KMP, Rabin-Karp, the Z-Algorithm and Aho-Corasick search the text for spam keywords and suspicious patterns.

The other algorithms support the analysis in different ways. Bitmask Dynamic Programming is used for feature-score calculation, Edmonds-Karp Maximum Flow and Greedy Set Cover Approximation are applied, randomized hashing uses a randomly selected base, and parallel processing handles several emails at the same time with multiple Java tasks. The detected features are combined into a spam score, and the classification is decided from this score. The system also gives the spam category, confidence score, risk level and URL and suspicious URL detection.

The web interface lets the user view the emails, run the analysis, compare the algorithms and read the detailed report in a browser. The project shows how DSA concepts such as string matching, dynamic programming, graph algorithms, greedy methods and hashing can be used together on a real problem.

The current system uses a rule/feature-based approach and does not use machine learning. No formal accuracy is claimed, because no labelled-dataset evaluation has been performed.

Future scope (this is future work and is not part of the current implementation):

Near-term: use larger email datasets, improve the classification rules and test the system more thoroughly.

Medium-term: better phishing URL analysis, database integration, user authentication and more advanced reports.

Long-term: machine learning integration, cloud deployment and integration with real email services.

9. REFERENCES

1. G. Shirvani and S. Ghasemshirazi, “Advancing Email Spam Detection: Leveraging Zero-Shot Learning and Large Language Models,” arXiv:2505.02362, 2025.
<https://arxiv.org/abs/2505.02362>
2. E. Hotoğlu, S. Sen and B. Can, “A Comprehensive Analysis of Adversarial Attacks against Spam Filters,” arXiv:2505.03831, 2025. <https://arxiv.org/abs/2505.03831>
3. A. A. Ali and A. A. Abdullah, “Email Spam Detection: A Novel Hybrid Approach Using Machine and Deep Learning Techniques,” International Journal of Intelligent Engineering and Systems, vol. 18, no. 7, 2025. <https://doi.org/10.22266/ijies2025.0831.12>
4. E. H. Tusher, M. A. Ismail and A. F. Mat Raffei, “Email Spam Classification Based on Deep Learning Methods: A Review,” Iraqi Journal for Computer Science and Mathematics, vol. 6, no. 1, pp. 24–36, 2025. <https://ijcsm.researchcommons.org/cgi/viewcontent.cgi?article=1236&context=ijcsm>
5. M. Kshirsagar, V. Rathi and C. Ryan, “Meta-learner-based frameworks for interpretable email spam detection,” Frontiers in Artificial Intelligence, vol. 8, 1569804, 2025.
<https://doi.org/10.3389/frai.2025.1569804>
6. Apache SpamAssassin Project. SpamAssassin public mail corpus.
<https://spamassassin.apache.org/old/publiccorpus/>
7. NIST. TREC Spam Track. <https://trec.nist.gov/data/spam.html>
8. CP-Algorithms. Prefix function and Knuth-Morris-Pratt algorithm.
<https://cp-algorithms.com/string/prefix-function.html>
9. CP-Algorithms. Aho-Corasick algorithm. https://cp-algorithms.com/string/aho_corasick.html
10. CP-Algorithms. Edmonds-Karp algorithm for maximum flow.
https://cp-algorithms.com/graph/edmonds_karp.html